Gestion des sessions

Complément de la séance 2 – simple, puis avec validité temporelle

Pour les étudiants ayant terminé l'exercice 4 de la séance 2

Environnement : XAMPP, VSCodium

1 PRÉSENTATION DU COMPLÉMENT

Ce complément s'adresse à celles et ceux qui ont terminé l'exercice 4 de la séance 2 (connexion et déconnexion). Vous avez déjà utilisé des sessions ; vous allez maintenant comprendre **comment elles fonctionnent**, les **durcir**, puis leur donner une **durée de validité** : une session qui expire après un temps d'inactivité, et une autre limite de durée absolue.

Durée indicative : 40 à 50 minutes.

Compétences visées

- Expliquer où sont stockées les données de session et ce que contient le cookie.
- Configurer le cookie de session de façon plus sûre (`HttpOnly`, `SameSite`, mode strict).
- Implémenter une **expiration par inactivité** et une **durée de vie maximale**, côté serveur.
- Renouveler périodiquement l'identifiant de session.
- Informer l'utilisateur de l'expiration (message, compte à rebours).

Prérequis :

Votre projet `tp-bdd` avec `SessionManager`, `index.php`, `dashboard.php`, `admin.php` et `logout.php` fonctionnels (séance 2, exercices 4 et 5).

Déroulé conseillé

Étape	Durée	Contenu
Partie 1 et exercice A	10 min	La session simple : observer le cookie et le fichier de session
Partie 2 et exercice B	10 min	Durcir le cookie de session
Partie 3 et exercices C et D	25 min	Expiration par inactivité, durée maximale, renouvellement de l'identifiant
Exercice E	bonus	Compte à rebours en JavaScript

2 PARTIE 1 – LA SESSION « SIMPLE »

2.1 COMMENT ÇA MARCHE

HTTP est sans mémoire : chaque requête repart de zéro (séance 1). Une **session** donne au serveur une mémoire par visiteur.

1. Au premier `session_start()`, PHP crée un **identifiant** aléatoire (par exemple `8slahgu0vmi275ae15hs12400h`).
2. Les données de la session (`$_SESSION`) sont stockées **sur le serveur**, dans un fichier nommé d'après l'identifiant (dossier indiqué par `session.save_path`, souvent `C:\xampp\tmp` avec XAMPP).
3. L'identifiant est envoyé au navigateur dans un **cookie** (par défaut `PHPSESSID`).
4. À chaque requête, le navigateur renvoie le cookie ; PHP retrouve le fichier et recharge `$_SESSION`.

	Où ?	Contenu
Cookie `PHPSESSID`	Navigateur	**Uniquement** l'identifiant
Fichier `sess_<identifiant>`	Serveur	Les données (`user_id`, `role`, `visites`…)

C'est pourquoi on peut stocker des informations de session sans que l'utilisateur les voie ou les modifie : seul l'identifiant circule. En revanche, **qui possède l'identifiant possède la session**. Toute la sécurité d'une session repose sur le secret de cet identifiant.

2.2 COMBIEN DE TEMPS DURE UNE SESSION ?

Par défaut :

- le cookie est un **cookie de session** : il disparaît à la fermeture du navigateur ;
- côté serveur, le fichier est supprimé par le ramasse-miettes de PHP (`session.gc_maxlifetime`, 1440 secondes par défaut), mais **de façon probabiliste** et seulement quand des requêtes arrivent. Cette purge sert à nettoyer le disque ; elle ne garantit **ni précision ni expiration à une heure donnée**.

Une session peut donc rester valide bien plus longtemps que prévu : onglet oublié sur un poste partagé, identifiant volé… Pour une application qui authentifie des utilisateurs, il faut **gérer soi-même la validité**, côté serveur.

3 PARTIE 2 – DURCIR LA SESSION

Ces réglages se font **avant** `session_start()` (après, PHP refuse de les modifier). Ils se placent dans le constructeur de `SessionManager`.

```
ini_set('session.use_strict_mode', '1');     // refuse un identifiant inconnu du serveur
ini_set('session.use_only_cookies', '1');    // jamais d'identifiant dans l'URL
session_set_cookie_params([
      lifetime' => 0,          // cookie de session : supprimé à la fermeture du navigateur
      'path'     => '/',
      'httponly' => true,       // invisible pour JavaScript
      'samesite' => 'Lax',      // non envoyé lors de requêtes provenant d'autres sites
]);
session_start();
```

Réglage	Protège contre
`use_strict_mode`	La **fixation de session** : un attaquant ne peut pas imposer un identifiant de son choix
`use_only_cookies`	La fuite de l'identifiant dans une URL (historique, journaux, lien partagé)
`httponly`	Le vol de l'identifiant par un script injecté (faille XSS) : `document.cookie` ne le voit pas
`samesite`	Une partie des attaques CSRF : le cookie n'est pas envoyé par un autre site
`secure`	L'interception du cookie sur le réseau : il n'est envoyé qu'en **HTTPS**

Le réglage `secure` est à activer dès que l'application est servie en HTTPS. Il est omis ici, car le TP tourne en HTTP sur `localhost` : avec `secure => true`, le cookie ne serait jamais renvoyé.

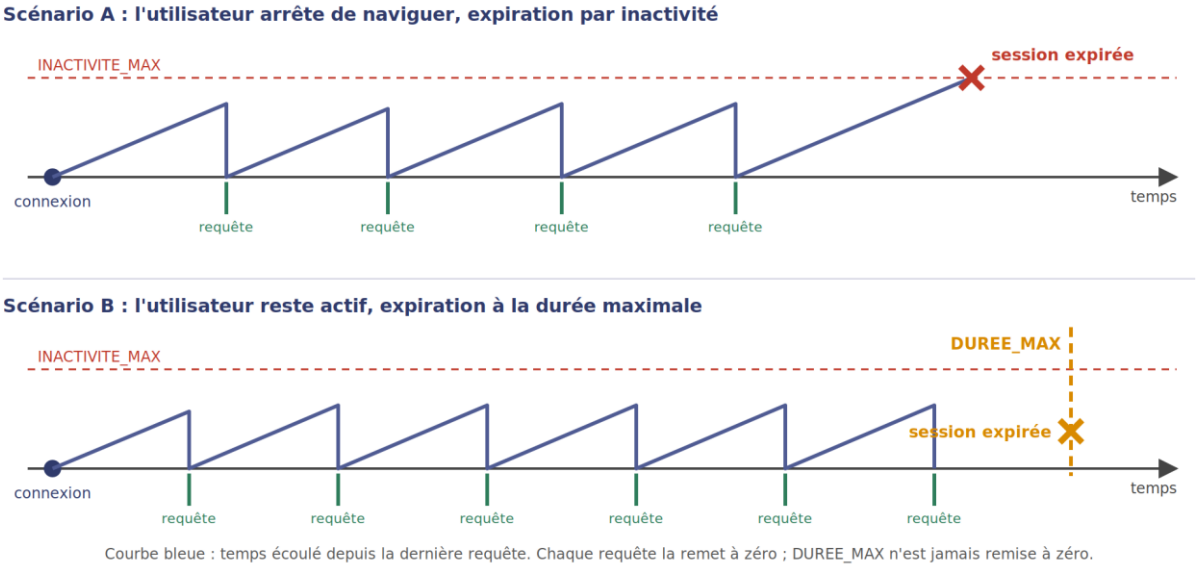
On y ajoute `session_regenerate_id(true)` à chaque **changement de privilège** (connexion, déconnexion) : l'ancien identifiant devient inutilisable.

4 PARTIE 3 – DONNER UNE DURÉE DE VALIDITÉ À LA SESSION

4.1 DEUX HORLOGES, DEUX PROTECTIONS

On définit deux limites, vérifiées **côté serveur à chaque requête** :

Limite	Question posée	Remise à zéro ?	Protège contre
Inactivité (`INACTIVITE_MAX`)	Combien de temps depuis la dernière requête ?	Oui, à chaque requête	Un poste laissé ouvert
Durée maximale (`DUREE_MAX`)	Combien de temps depuis la connexion ?	**Jamais**	Un identifiant volé qui serait utilisé en continu



Les deux limites : inactivité (remise à zéro par chaque requête) et durée maximale

À titre indicatif, une application sensible choisit une inactivité de l'ordre de la dizaine de minutes ; une application courante de quelques dizaines de minutes ; la durée maximale se compte en heures (une journée de travail, par exemple). Le bon choix dépend de la sensibilité des données et du confort attendu.

4.2 L'ALGORITHME À CHAQUE REQUÊTE

1. Il n'y a pas d'utilisateur connecté : la session n'est pas valide.
2. Temps écoulé depuis la dernière activité > `INACTIVITE_MAX` : **expiration**.
3. Temps écoulé depuis la connexion > `DUREE_MAX` : **expiration**.
4. Sinon : la requête compte comme une activité, on enregistre l'heure actuelle.
5. Si l'identifiant n'a pas été renouvelé depuis `RENOUVELLEMENT` secondes : `session_regenerate_id(true)`.

On compare des **horodatages Unix** obtenus avec `time()` (secondes depuis le 1er janvier 1970, UTC) : ils ne dépendent donc ni du fuseau horaire ni de l'heure affichée par le navigateur.

4.3 LE SESSIONMANAGER COMPLET

```php
<?php
declare(strict_types=1);

final class SessionManager
{
    /** Déconnexion après ce délai sans aucune activité (secondes). */
    public const INACTIVITE_MAX = 900;       // 15 minutes
    /** Durée de vie maximale d'une connexion, même si l'utilisateur reste actif (secondes). */
    public const DUREE_MAX = 28800;          // 8 heures
    /** Délai entre deux changements de l'identifiant de session (secondes). */
    public const RENOUVELLEMENT = 300;       // 5 minutes

    private static ?self $instance = null;

    private function __construct()
    {
        if (session_status() === PHP_SESSION_NONE) {
            ini_set('session.use_strict_mode', '1');     // refuse un identifiant inconnu du
serveur
            ini_set('session.use_only_cookies', '1');    // jamais d'identifiant dans l'URL
            session_set_cookie_params([
                'lifetime' => 0,          // cookie de session : supprimé à la fermeture du
navigateur
                'path'     => '/',
                'httponly' => true,       // invisible pour JavaScript (limite le vol par XSS)
                'samesite' => 'Lax',      // non envoyé lors de requêtes provenant d'autres sites
            ]);
            session_start();
        }
    }

    public static function getInstance(): self
    {
        return self::$instance ??= new self();
    }

    public function set(string $cle, mixed $valeur): void
    {
        $_SESSION[$cle] = $valeur;
    }

    public function get(string $cle): mixed
    {
        return $_SESSION[$cle] ?? null;
    }

    public function has(string $cle): bool
    {
        return isset($_SESSION[$cle]);
    }

    /** Ouvre une session authentifiée : nouvel identifiant et horodatages de départ. */
    public function ouvrir(array $donnees): void
    {
        session_regenerate_id(true);
        $maintenant = time();
        $_SESSION = $donnees + [
            '_debut'     => $maintenant,    // moment de la connexion (durée maximale)
            '_activite'  => $maintenant,    // dernière requête (inactivité)
            '_renouvele' => $maintenant,    // dernier changement d'identifiant
        ];
    }

    /** Vrai si un utilisateur est connecté ET que sa session n'a pas expiré. */
    public function estValide(): bool
    {
```

```
        if (!isset($_SESSION['user_id'], $_SESSION['_debut'], $_SESSION['_activite'])) {
            return false;
        }

        $maintenant = time();

        if ($maintenant - (int) $_SESSION['_activite'] > self::INACTIVITE_MAX) {
            $this->expirer('Votre session a expiré après une période d\'inactivité.');
            return false;
        }
        if ($maintenant - (int) $_SESSION['_debut'] > self::DUREE_MAX) {
            $this->expirer('Votre session a atteint sa durée maximale : reconnectez-vous.');
            return false;
        }

        $_SESSION['_activite'] = $maintenant;     // la requête compte comme une activité

        if ($maintenant - (int) ($_SESSION['_renouvele'] ?? 0) > self::RENOUVELLEMENT) {
            session_regenerate_id(true);
            $_SESSION['_renouvele'] = $maintenant;
        }
        return true;
    }

    /** Secondes avant l'expiration la plus proche (inactivité ou durée maximale). */
    public function secondesRestantes(): int
    {
        $maintenant = time();
        $inactivite = self::INACTIVITE_MAX - ($maintenant - (int) ($_SESSION['_activite'] ?? 0));
        $duree      = self::DUREE_MAX - ($maintenant - (int) ($_SESSION['_debut'] ?? 0));
        return max(0, min($inactivite, $duree));
    }

    /** Vide la session mais la garde ouverte, pour pouvoir expliquer pourquoi à l'utilisateur.
*/
    private function expirer(string $message): void
    {
        session_regenerate_id(true);
        $_SESSION = ['flash' => $message];
    }

    public function setFlash(string $message): void
    {
        $_SESSION['flash'] = $message;
    }

    /** Lit le message puis le supprime : il ne s'affiche qu'une fois. */
    public function popFlash(): ?string
    {
        $message = $_SESSION['flash'] ?? null;
        unset($_SESSION['flash']);
        return is_string($message) ? $message : null;
    }

    /** Déconnexion volontaire : vide la session, supprime le cookie et détruit les données. */
    public function detruire(): void
    {
        $_SESSION = [];
        if (ini_get('session.use_cookies')) {
            $p = session_get_cookie_params();
            setcookie(session_name(), '', [
                'expires'  => time() - 42000,
                'path'     => $p['path'],
                'domain'   => $p['domain'],
                'secure'   => $p['secure'],
                'httponly' => $p['httponly'],
                'samesite' => $p['samesite'] ?: 'Lax',
            ]);
        }
```

```
        session_destroy();
    }

    private function __clone() {}

    public function __wakeup(): void
    {
        throw new LogicException('Un singleton ne peut pas être désérialisé.');
    }
}
```

Points à comprendre :

- `ouvrir()` remplace le `regenerer()` suivi de plusieurs `set()` de la séance 2 : il crée le nouvel identifiant et enregistre les trois horodatages (`_debut`, `_activite`, `_renouvele`).
- `estValide()` remplace le test `has('user_id')` dans toutes les pages protégées. **C'est elle qui fait vivre ou mourir la session.**
- `expirer()` vide la session mais la **garde ouverte** : on a besoin d'un support (`$_SESSION`) pour y mettre le message « Votre session a expiré », que `index.php` affichera une seule fois. `detruire()` ne servirait à rien ici, car il supprime tout.
- `secondesRestantes()` renvoie le temps avant la prochaine expiration (utile pour informer l'utilisateur).
- Les valeurs des constantes (15 minutes, 8 heures, 5 minutes) sont réalistes ; **pour tester, vous les réduirez** à quelques secondes.

4.4 UTILISATION DANS LES PAGES

Dans `index.php`, la connexion réussie devient :

```
$utilisateur = Utilisateur::depuisLigne($user);
$session->ouvrir([
    'user_id'  => $utilisateur->id,
    'username' => $utilisateur->username,
    'role'     => $utilisateur->role->value,
]);
```

Dans `index.php` (utilisateur déjà connecté), `dashboard.php` et `admin.php`, le test de connexion devient :

```
if (!$session->estValide()) {          // dans index.php : if ($session->estValide()) {
...vers le tableau de bord... }
    header('Location: index.php');
    exit;
}
```

> **Pourquoi ne pas se contenter de la durée de vie du cookie ?** Le cookie appartient au client : il peut être copié, rejoué ou prolongé. Une date d'expiration n'a de valeur que si c'est le **serveur** qui la vérifie.

5 TP COMPLÉMENTAIRE

5.1 EXERCICE A – OBSERVER UNE SESSION SIMPLE (10 MIN)

Objectif : voir de ses yeux l'identifiant, le cookie et le fichier de session.

- Créer `compteur.php`, qui compte le nombre de chargements de la page **dans la session** :

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/fonctions.php';
require_once __DIR__ . '/SessionManager.php';

$session = SessionManager::getInstance();

$visites = (int) ($session->get('visites') ?? 0) + 1;
$session->set('visites', $visites);
?>
<!DOCTYPE html>
<html lang="fr">
<head><meta charset="utf-8"><title>Compteur de visites</title></head>
<body>
  <h1>Compteur de visites</h1>
  <p>Cette session a chargé cette page <strong><?= $visites ?></strong> fois.</p>
  <p>Cookie <code><?= h(session_name()) ?></code> : <code><?= h(substr(session_id(), 0,
8)) ?>…</code></p>
  <p><a href="compteur.php">Recharger</a></p>
</body>
</html>
```

- Ouvrir `http://localhost:8000/compteur.php` et recharger plusieurs fois.
- Ouvrir les outils de développement (`F12`) : onglet **Stockage** (Firefox) ou **Application** (navigateurs basés sur Chromium, comme Vivaldi), rubrique **Cookies**. Repérer `PHPSESSID`.
- Dans l'explorateur, ouvrir le dossier des sessions (`session.save_path`, souvent `C:\xampp\tmp`) et repérer le fichier `sess_...` qui correspond à votre identifiant. L'ouvrir avec VSCodium : que contient-il ?
- Ouvrir la même page dans une **fenêtre de navigation privée** ou un autre navigateur.
- Supprimer le cookie `PHPSESSID` depuis les outils de développement, puis recharger.

À vérifier

- ☐ Le compteur augmente à chaque rechargement ; dans la navigation privée, il repart de 1.
- ☐ Le fichier de session contient les données (`visites|i:...;`) mais **pas** votre mot de passe ni rien d'autre que ce que le script y a mis.
- ☐ Après suppression du cookie, le compteur repart de 1. Pourquoi ? Que devient l'ancien fichier de session ?

5.2 EXERCICE B – DURCIR LE COOKIE (10 MIN)

Objectif : appliquer les réglages de la partie 2.

- Dans le constructeur de `SessionManager`, ajouter les deux `ini_set` et `session_set_cookie_params` **avant** `session_start()`.
- Recharger `compteur.php` et regarder les attributs du cookie dans les outils de développement : `HttpOnly` doit être coché, `SameSite` doit valoir `Lax`.
- Dans la console du navigateur, taper `document.cookie` : `PHPSESSID` ne doit plus apparaître.
- **Test du mode strict :** dans les outils de développement, remplacer la valeur du cookie `PHPSESSID` par un texte inventé (`abc123`), puis recharger.

À vérifier

- ☐ Le cookie porte les attributs `HttpOnly` et `SameSite=Lax`.
- ☐ `document.cookie` ne montre pas l'identifiant de session.
- ☐ Avec un identifiant inventé, le serveur **n'accepte pas** `abc123` : il crée une nouvelle session (le compteur repart de 1 et le cookie change). Que se passerait-il sans `use_strict_mode` ?

5.3 EXERCICE C – EXPIRATION PAR INACTIVITÉ (15 MIN)

Objectif : déconnecter l'utilisateur qui ne fait plus rien.

1. Dans `SessionManager`, ajouter la constante `INACTIVITE_MAX` et les méthodes `ouvrir()`, `estValide()` (sans la partie durée maximale, pour l'instant) et `expirer()`. Pour tester, fixer **`INACTIVITE_MAX = 20`** secondes.
2. Dans `index.php`, utiliser `ouvrir()` au lieu de `regenerer()` suivi des `set()`.
3. Dans `index.php`, `dashboard.php` et `admin.php`, remplacer `has('user_id')` par `estValide()`.
4. S'assurer que `index.php` affiche le message du `popFlash()`.

À vérifier

- ☐ Après connexion, recharger `dashboard.php` toutes les 10 secondes : la session reste ouverte (**chaque requête repousse l'expiration**).
- ☐ Attendre plus de 20 secondes sans rien faire puis recharger : retour sur `index.php` avec le message « Votre session a expiré après une période d'inactivité. »
- ☐ Le message ne s'affiche qu'une fois.
- ☐ Après expiration, taper directement l'adresse de `admin.php` : l'accès est refusé (redirection vers la connexion).

5.4 EXERCICE D – DURÉE MAXIMALE ET RENOUVELLEMENT DE L'IDENTIFIANT (15 MIN)

Objectif : limiter la durée d'une connexion, même pour un utilisateur actif, et changer régulièrement d'identifiant.

1. Ajouter à `estValide()` le test de la **durée maximale** (`_debut`) et la constante `DUREE_MAX`. Pour tester : **`DUREE_MAX = 60`** secondes (et `INACTIVITE_MAX` à 20).
2. Ajouter le **renouvellement** périodique de l'identifiant (`RENOUVELLEMENT = 15` secondes pour le test).
3. Tester : se connecter, puis recharger `dashboard.php` environ toutes les 10 secondes, en surveillant la valeur du cookie `PHPSESSID` dans les outils de développement.
4. **Remettre les valeurs réalistes** à la fin : 900, 28800 et 300 secondes.

À vérifier

- ☐ Malgré une activité continue, la session expire au bout d'environ 60 secondes, avec le message « durée maximale ».
- ☐ L'identifiant `PHPSESSID` change environ toutes les 15 secondes, sans que vous soyez déconnecté.
- ☐ Question : pourquoi renouveler l'identifiant alors que l'utilisateur est légitime ? (Indice : que se passe-t-il pour un attaquant qui aurait copié l'ancien identifiant ?)
- ☐ Les constantes ont retrouvé leurs valeurs réalistes.

5.5 EXERCICE E (BONUS) – COMPTE À REBOURS EN JAVASCRIPT

Objectif : prévenir l'utilisateur avant l'expiration.

- Dans `dashboard.php`, afficher « Expiration de la session dans X min YY s » grâce à `secondesRestantes()`. Le serveur transmet la valeur au script via un attribut `data-secondes`, et le JavaScript décrémente l'affichage chaque seconde. À la fin du compte à rebours, la page redirige vers `index.php`.
- Attention : **le compte à rebours n'est qu'un confort d'affichage**. C'est le serveur qui décide ; le navigateur ne fait que l'afficher.
- Question : ouvrez `dashboard.php` dans deux onglets. Que constatez-vous sur le compte à rebours de l'onglet que vous n'utilisez pas ? Pourquoi ?

Indice : le script de redirection doit attendre **une seconde de plus** que la fin du compte à rebours, sinon le serveur peut considérer la session comme encore valide et la prolonger.

6 ANNEXES

6.1 AIDE-MÉMOIRE

Besoin	Code
Démarrer la session	`session_start()`
Régler le cookie (avant `session_start`)	`session_set_cookie_params([...])`
Nouvel identifiant (connexion, renouvellement)	`session_regenerate_id(true)`
Heure actuelle (secondes, UTC)	`time()`
Écart avec la dernière activité	`time() - $_SESSION['_activite']`
Vider la session	`$_SESSION = [];`
Supprimer la session côté serveur	`session_destroy()`
Supprimer le cookie	`setcookie(session_name(), '', ['expires' => time() - 42000, ...])`
Dossier des fichiers de session	`session_save_path()` ou `php -i` (ligne `session.save_path`)

6.2 ERREURS FRÉQUENTES ET DIAGNOSTIC

Symptôme	Cause probable	Solution
`Session cookie parameters cannot be changed when a session is active`	`session_set_cookie_params()` appelé après `session_start()`	Le placer avant `session_start()` (dans le constructeur, sous la condition `PHP_SESSION_NONE`)
La session expire immédiatement	`INACTIVITE_MAX` trop petit, ou `_activite` jamais défini (connexion sans `ouvrir()`)	Utiliser `ouvrir()` à la connexion
La session n'expire jamais	Pages qui testent encore `has('user_id')`	Remplacer par `estValide()` dans **toutes** les pages protégées
Le message d'expiration n'apparaît pas	`index.php` redirige avant de lire le message, ou n'appelle pas `popFlash()`	Utiliser `estValide()` dans `index.php` et afficher `popFlash()`
Boucle de redirections	`dashboard.php` et `index.php` se renvoient l'un à l'autre	Vérifier que `index.php` ne redirige que si `estValide()` est vrai
Le cookie n'est jamais renvoyé	`'secure' => true` alors que le site est en HTTP	Ne l'activer qu'en HTTPS
Déconnecté à chaque requête	Cookies bloqués, ou `session_regenerate_id` appelé à chaque page	Vérifier le cookie dans les outils de développement ; renouveler seulement après `RENOUVELLEMENT` secondes
Délais de test restés en place	Constantes à 20 ou 60 secondes	Remettre 900, 28800 et 300

6.3 POUR ALLER PLUS LOIN

- **HTTPS et cookie `secure` :** indispensables en production ; sans HTTPS, l'identifiant circule en clair.
- **Stocker les sessions en base de données** (`session_set_save_handler`) pour les partager entre plusieurs serveurs et pouvoir **lister ou révoquer** les sessions d'un utilisateur.
- **« Se souvenir de moi » :** un second jeton, durable, stocké **haché** en base et renouvelé à chaque usage ; à ne jamais confondre avec l'identifiant de session.
- **Jetons CSRF** pour les formulaires et la déconnexion.
- **Journalisation** des connexions, des expirations et des accès refusés avec le `Logger`.
- **Références :** la documentation PHP sur les sessions (`php.net/manual/fr/book.session.php`).